

Como um modelo aprende

Quais dados, objetivos e decisões transformam
parâmetros aleatórios em um modelo de linguagem

o nosso modelo, em quatro momentos do treino

passo 0 2cn%4Σ+1—oMbvA3Çtbàf√8f_Nl<Íé1YB,ôT3s;ô: %:JZõxK"5≈

passo 300 fi br consegue vesste nde to paite s te fa se a ver a nina a

passo 600 Me frales do ergergum flasto e com pera em por sógra. Maca a

passo 2000 token modem uma frase e com e valor um LLM e colem 2 é amede

texto gerado de verdade, não ilustração

A pergunta

Um modelo recém-criado só produz ruído. Semanas depois, escreve. O que acontece no meio é pretraining — e ele é feito de dados, de um objetivo e de decisões.

Conteúdo da seção 3

- corpus, limpeza e validação
- labels deslocados
- entropia cruzada e perplexidade
- lote, época, passo e taxa
- curvas e overfitting
- checkpoints, sementes, orçamento

Estrutura da apresentação

1

Pergunta e motivação

O que muda entre um modelo recém-criado e um modelo treinado.

2

Intuição e mapa conceitual

O laço de treinamento inteiro em um diagrama.

3

Explicação técnica

Dados, objetivo, otimização, diagnóstico e orçamento — com números do nosso treino.

nosso conteúdo mínimo

4

Visualização e artefato

Um modelo de linguagem treinado do zero, ao vivo.

5

Literatura e projeto

Os artigos que definiram as regras e a ponte para fine-tuning.

6

Síntese e limites

O que o pretraining entrega e o que ele não garante.

1

Pergunta e motivação

O que acontece entre o primeiro e o último passo de treino.

- A pergunta orientadora
- Assistindo um modelo aprender
- Por que as decisões pesam

A pergunta orientadora

Quais dados, objetivos e decisões transformam parâmetros aleatórios em um modelo de linguagem?

1 Dados

o que o modelo lê

Corpus, limpeza, mistura das fontes e a separação entre o texto que ensina e o texto que avalia.

corpus → lotes

2 Objetivo

o que o modelo tenta fazer

Prever o próximo token em cada posição.
A entropia cruzada mede o quanto ele erra; o gradiente diz como errar menos.

$\min -\ln P(\text{próximo})$

3 Decisões

como o treino é conduzido

Taxa de aprendizado, tamanho do lote e do modelo, quando parar, o que salvar e quanto computação gastar.

$\eta \cdot B \cdot N \cdot D \cdot \text{passos}$

As três palavras da pergunta organizam a seção 3.

Assistindo um modelo aprender

Treinamos um modelo de linguagem para este seminário. Estas são amostras reais que ele gerou ao longo do treino.

passo	texto gerado (trecho)	perda de validação
0	2cn%4Σ+1—oMbvA3Çtbàf√8f_Nl<Íé1YB,ôT3s;ô:?:JZõxK"5≈Vu)%ãlFG-7%áY0	4,74
100	Der ;r :áoEongesentotengo cos e ,o iras eito. ó r -uo d, (u	2,85
300	fi br consegue vesste nde to paite s te fa se a ver a nina ara i a	2,60
600	Me frales do ergergum flasto e com pera em por sógra. Maca a las	2,49
1100	Mena sabtecos conteito e o preinar a mina e sem do s bula co vol	<i>melhor</i> 2,31
2000	token modem uma frase e com e valor um LLM e colem 2 é amedes ol	2,37
6000	A resposta fechada e as trincas que ele aprende Uuma Tuan) e rec	<i>decorou</i> 2,73

A contradição que a seção 3 resolve:

a última linha é o texto mais fluente da tabela — e tem validação pior que a do passo 300. 65% da amostra é cópia literal do corpus de treino.

Por que as decisões de pretraining pesam tanto

1

Custo

Treinar um LLM ocupa milhares de aceleradores por semanas. Um erro de configuração descoberto no meio do caminho custa esse tempo inteiro.

2

Irreversibilidade

Não existe desfazer: só retomar de um checkpoint salvo. Por isso as decisões são testadas antes, em modelos pequenos — como o nosso.

3

Origem do conhecimento

Tudo o que o modelo sabe entra aqui. O fine-tuning, tema da nossa equipe, apenas ajusta o que o pretraining construiu.

$3,14 \times 10^{23}$

operações de ponto flutuante para treinar o GPT-3

Para comparar: o nosso modelo inteiro custou $3,4 \times 10^{11}$ operações — quase um trilhão de vezes menos. Cabe num notebook e treina em poucos minutos, num computador comum. É por isso que dá para errar, medir e errar de novo: exatamente o que um LLM de verdade não pode fazer.

GPT-3: Brown et al. (2020). Nosso modelo: $6 \times \text{parâmetros} \times \text{tokens processados}$.

2

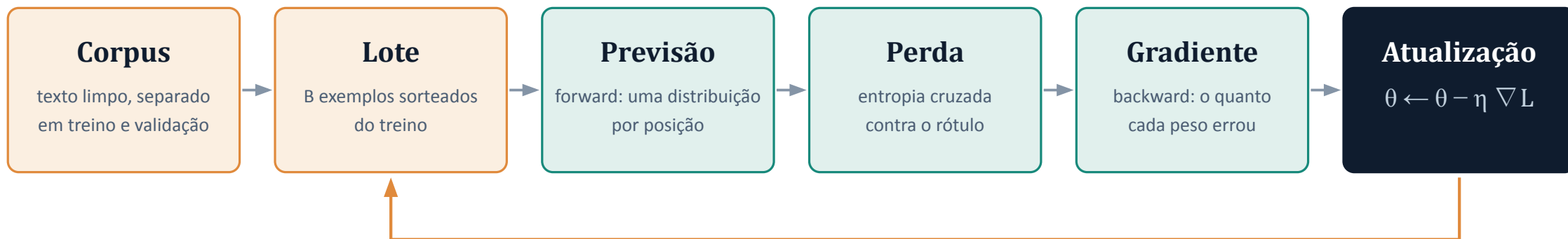
Intuição e mapa conceitual

O laço inteiro antes das fórmulas.

- O laço de treinamento
- Três intuições que bastam

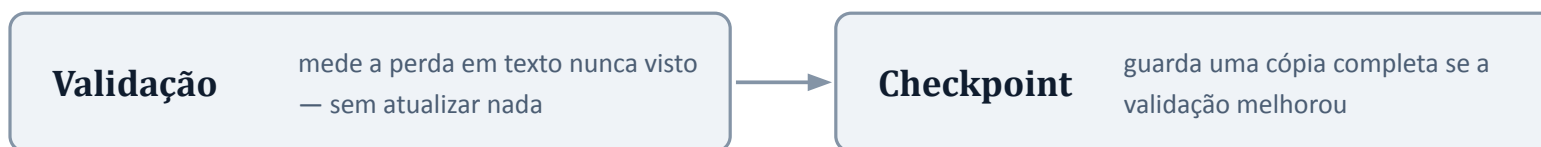
O mapa: o laço de treinamento

Todo o pretraining é este ciclo, repetido. Cada caixa é um slide da seção 3.



próximo lote — cada volta é um passo; o GPT-3 deu cerca de 94 mil, o nosso modelo 6 mil

a cada 50 passos, fora do laço:



Leia assim:

os dados entram pela esquerda, a matemática acontece no meio e o único efeito do ciclo é mudar um pouco os pesos. Nada é programado à mão: tudo o que o modelo 'sabe' é o acúmulo dessas pequenas mudanças.

Três intuições que sustentam o resto

1

Um palpiteiro corrigido bilhões de vezes

Em cada posição o modelo dá um palpite sobre o próximo token e é corrigido na hora. O conhecimento é o resíduo acumulado dessas correções.

palpite → erro → ajuste

2

A perda é a medida da surpresa

Quanto menos probabilidade o modelo deu ao token que de fato veio, maior a perda. Treinar é tornar o texto real cada vez menos surpreendente.

perda = $-\ln P(\text{certo})$

3

Aprender é descer uma ladeira

Cada peso é uma direção num espaço enorme. O gradiente aponta a subida mais íngreme da perda; o treino dá um passo no sentido oposto.

$\theta \leftarrow \theta - \eta \nabla L$

3

Explicação técnica

Cada decisão do laço, com números do nosso treino.

- Corpus e limpeza
- Amostragem e mistura
- Treino × validação
- Labels deslocados
- Entropia cruzada e perplexidade
- Lote, passo, época e gradiente
- Taxa de aprendizado
- Curvas e overfitting
- Checkpoints, sementes, orçamento

Corpus e limpeza: o que o modelo vai ler

Texto bruto da internet é, em sua maior parte, inútil para treino. A limpeza decide o que o modelo vai aprender.

01	Coleta	rastreamento da web (Common Crawl), livros, código, artigos
02	Extração	tirar o texto do HTML: descartar menus, anúncios e scripts
03	Idioma	um classificador de língua mantém só o que interessa
04	Qualidade	heurísticas: tamanho da linha, pontuação, palavras proibidas
05	Deduplicação	remover cópias exatas e quase-cópias de documentos
06	Tokenização	o texto que sobrou vira IDs — o tema da semana 2

NO NOSSO CORPUS (MEDIDO)

33.329 → 28.662

caracteres antes e depois da limpeza (86% mantidos)

333 linhas com menos de 5 palavras

93 boilerplate: repetidas em 3+ documentos

12 duplicatas exatas

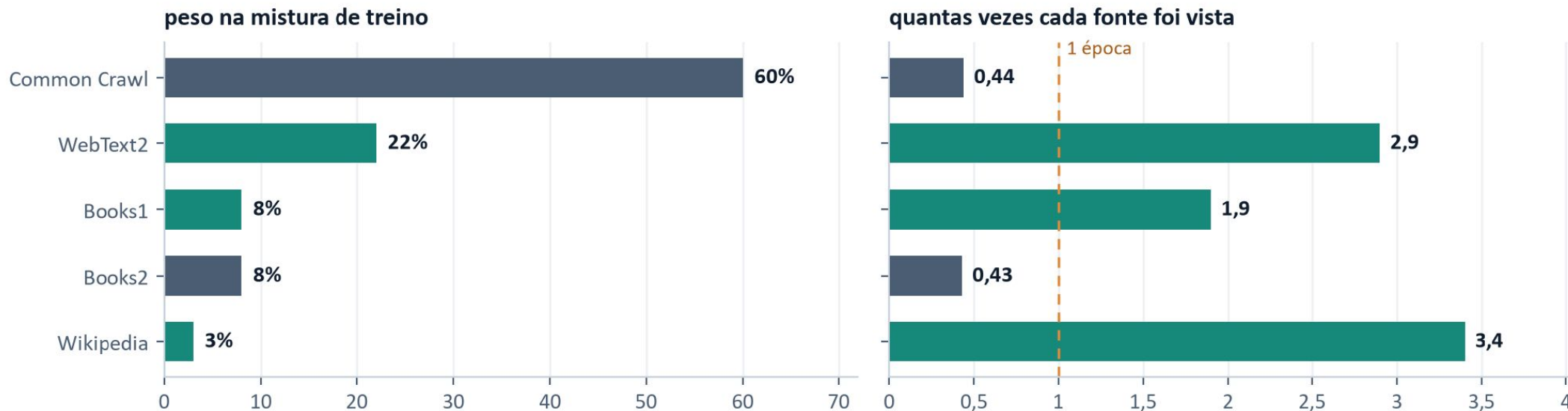
1 linha de código

O rodapé “3 · Explicação técnica” estava em 14 dos 33 documentos — o equivalente, no nosso corpus, aos menus e rodapés que se repetem em todo site.

Heurísticas no estilo do C4 (Raffel et al., 2020): linhas com ≥ 5 palavras, sem código, sem boilerplate, sem duplicatas.

Amostragem: o corpus não é lido em proporção ao tamanho

Cada fonte recebe um peso na mistura. Fontes de qualidade são repetidas; a web bruta é subamostrada.



Qualidade ganha peso

A Wikipedia é só 3% da mistura, mas foi vista 3,4 vezes durante o treino do GPT-3.

Volume é subamostrado

O Common Crawl é 60% dos exemplos e foi visto menos de meia vez: a maior parte nunca entrou num lote.

A mistura é uma decisão

Não existe proporção natural. Os pesos são hiperparâmetros escolhidos antes do treino — e caros de corrigir depois.

Dados do GPT-3: Brown et al. (2020), tabela 2.2 — 300 bilhões de tokens de treino.

Treino × validação: o termômetro honesto

A validação mede o que o modelo faz com texto que nunca viu. Se ela vaza para o treino, o termômetro mente.

os 33 documentos do nosso corpus

1	2	3	4	5	6	7	8	9	10	11
12	13	14	15	16	17	18	19	20	21	22
23	24	25	26	27	28	29	30	31	32	33

■ treino: 28 documentos · 24.225 exemplos

■ validação: 5 documentos · 4.470 exemplos

Sorteio por documento, com semente fixa (42): quem foi para a validação não tem nenhuma linha no treino.

Separe por documento, não por linha

Linhas do mesmo documento se parecem demais. Se metade treina e metade valida, o modelo reconhece o assunto em vez de generalizar — e a validação sai otimista.

A validação nunca gera gradiente

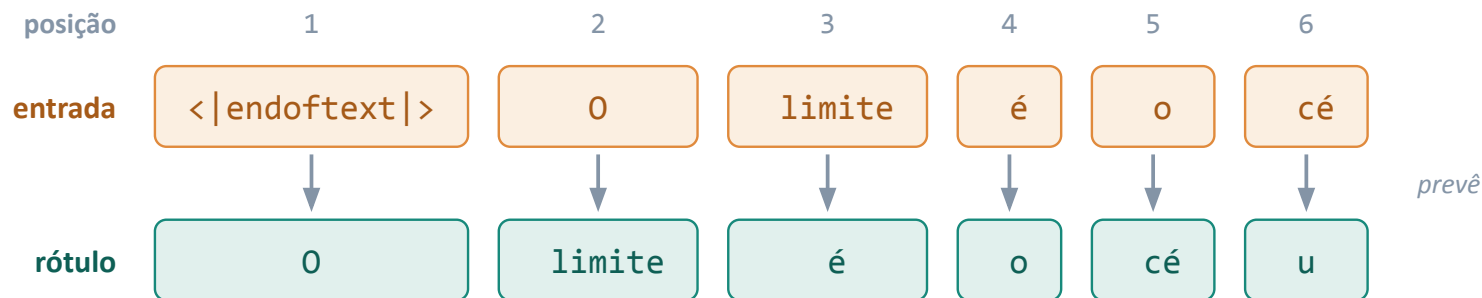
Ela só é medida. No nosso laço, a cada 50 passos calculamos a perda nos 4.470 exemplos de validação sem mudar um único peso.

Cuidado com contaminação

Em escala de internet, questões de provas e benchmarks aparecem no corpus. Um modelo pode acertar a prova por tê-la lido — e a nota deixa de medir capacidade.

Sequências de entrada e labels deslocados

No pretraining não existe rótulo humano: o rótulo de cada posição é simplesmente o token seguinte.



a linha de baixo é a de cima deslocada uma posição para a esquerda

Uma frase, n exemplos

Seis tokens viram seis previsões treinadas num único forward: a máscara causal da semana 2 impede que cada posição veja o próprio rótulo.

Documentos são concatenados

O corpus vira um fluxo contínuo, separado por <|endoftext|> (ID 100257 no tokenizer do GPT-4) e cortado em blocos do tamanho do contexto.

No nosso modelo: caracteres

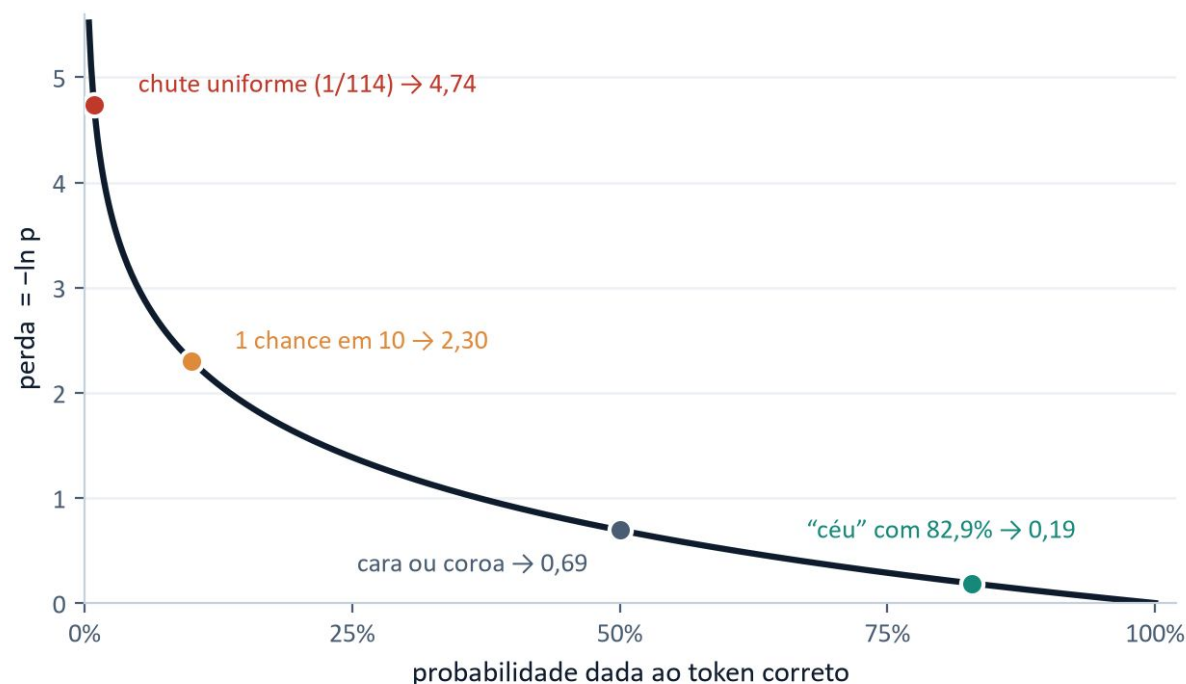
Os 8 caracteres anteriores preveem o próximo; ¶ faz o papel do <|endoftext|>.

¶¶¶¶¶¶¶¶	→	o	
¶¶¶¶¶¶¶O	→	.	
¶¶¶¶¶¶¶O.	→	m	· = espaço

Entropia cruzada: a nota de cada previsão

perda = -média de $\ln P(\text{token correto} \mid \text{contexto})$

uma parcela por posição, somando o lote inteiro



Por que o logaritmo

Pune pouco quem acerta com confiança e muito quem erra com confiança. Dar 1% ao token certo custa 4,61; dar 83% custa só 0,19.

O teste de sanidade

Um modelo recém-criado chuta uniforme: perda = $\ln |V|$. Nosso vocabulário tem 114 caracteres, então a teoria prevê $\ln 114 = 4,736$. No passo 0 medimos 4,738.

Ligação com a semana 2

É o softmax do slide 22 visto de outro ângulo: a perda só olha a probabilidade que foi dada ao token certo.

Perplexidade: a perda numa escala que dá para imaginar

$$\text{perplexidade} = e^{\text{perda}}$$

mesma informação, outra unidade

Interpretação: o modelo está tão indeciso quanto se escolhesse ao acaso entre PPL opções igualmente prováveis.

114,2

no passo 0

tão indeciso quanto sortear entre os 114 caracteres do vocabulário. É o chute uniforme.

9,05

nosso melhor modelo

como escolher entre cerca de 9 caracteres. O treino eliminou mais de 90% das opções em cada posição.

1

o limite teórico

previsão perfeita: nunca atingida em texto real, porque a própria língua tem incerteza.

Cuidado ao comparar

Perplexidade só se compara com o mesmo tokenizer. Um caractere, uma subpalavra e uma palavra carregam quantidades diferentes de informação: a perplexidade do nosso modelo de caracteres e a do GPT-4 não estão na mesma unidade (semana 2, slide 29).

Lote, passo e época: quanto dado o modelo já viu

Lote batch

Quantos exemplos entram em cada atualização. O gradiente é a média sobre o lote.

nosso: 64 exemplos

GPT-3: 3,2 milhões de tokens

Passo step

Uma atualização dos pesos: um forward, um backward e um ajuste.

nosso: 6.000 passos

GPT-3: cerca de 94 mil

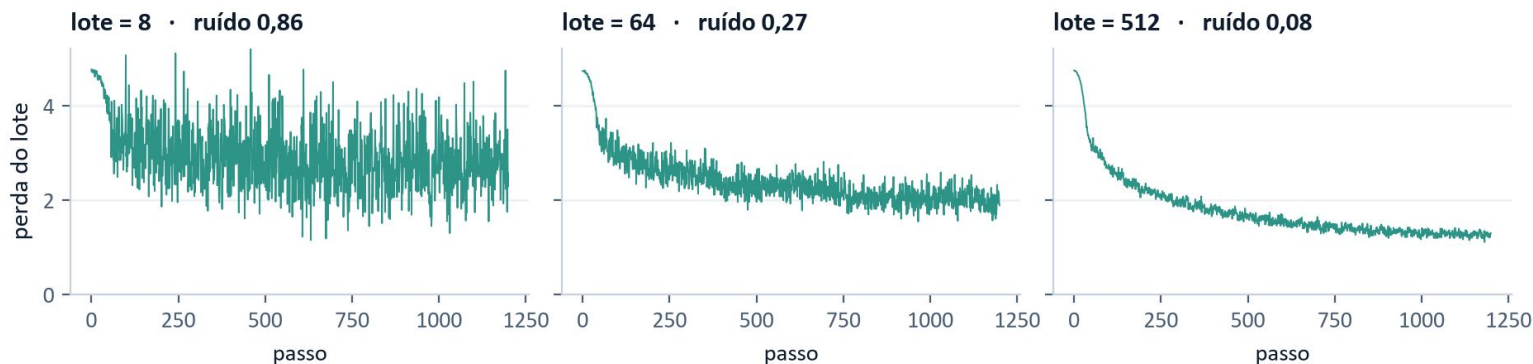
Época epoch

Uma passada completa por todo o conjunto de treino.

nosso: 378 passos por época → 15,9 épocas

GPT-3: menos de 1 época na web

O tamanho do lote e o ruído — mesmo modelo, mesmos 1.200 passos:



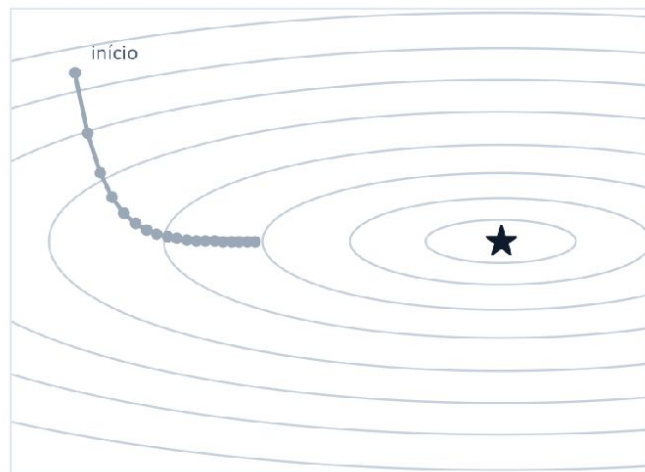
Lote maior dá um gradiente menos ruidoso ($0,86 \rightarrow 0,08$, cerca de $1/\sqrt{B}$) — mas cada passo com 512 exemplos custa 64 vezes mais que com 8.

Gradient descent: um passo contra a subida

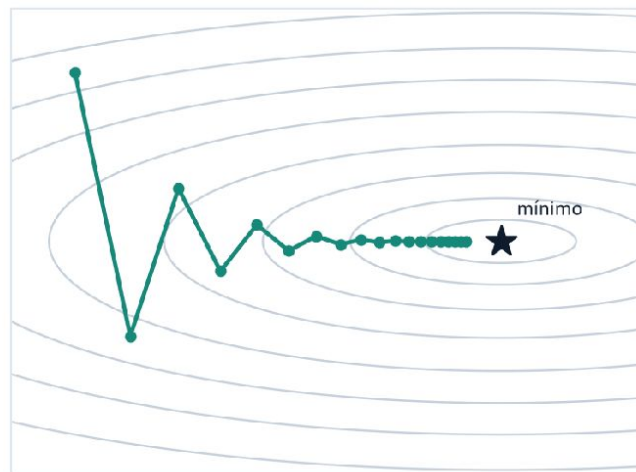
$$\theta \leftarrow \theta - \eta \cdot \nabla \theta L$$

η é a taxa de aprendizado; o gradiente aponta a subida da perda

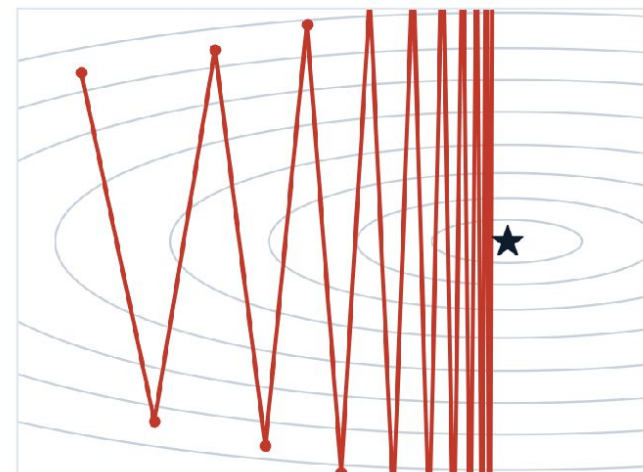
η pequeno: passos tímidos



η adequado: converge



η grande: oscila e diverge



1. forward

```
perda = modelo.perda(X, Y)  
previsões do lote e entropia cruzada
```

2. backward

```
g = modelo.gradientes(Y)  
quanto cada peso contribuiu para o erro
```

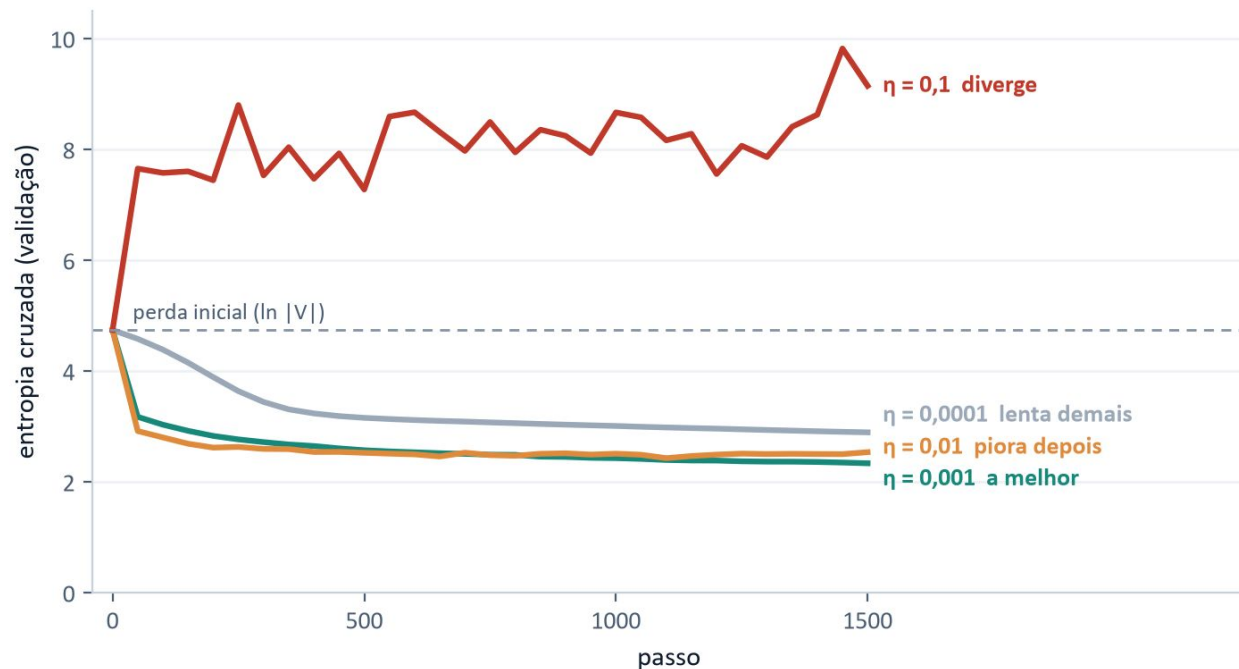
3. atualização

```
opt.passo(pesos, g,  $\eta$ )  
move cada peso contra o próprio gradiente
```

Superfície de brinquedo com 2 parâmetros. O nosso modelo tem 147.738 — mesma ideia, mais dimensões. Gradientes conferidos por diferenças finitas: erro relativo $3,6 \times 10^{-10}$.

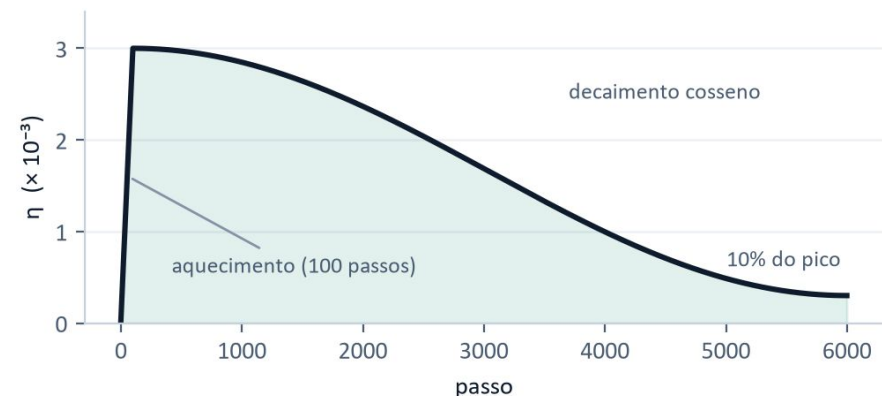
Taxa de aprendizado: a decisão mais sensível

Treinamos o mesmo modelo quatro vezes, mudando apenas η . Tudo o mais ficou fixo.



Com $\eta = 0,1$ a perda termina em 9,1 — pior que o chute inicial de 4,74. O modelo desaprendeu.

O que usamos no treino principal



Aquecimento + cosseno

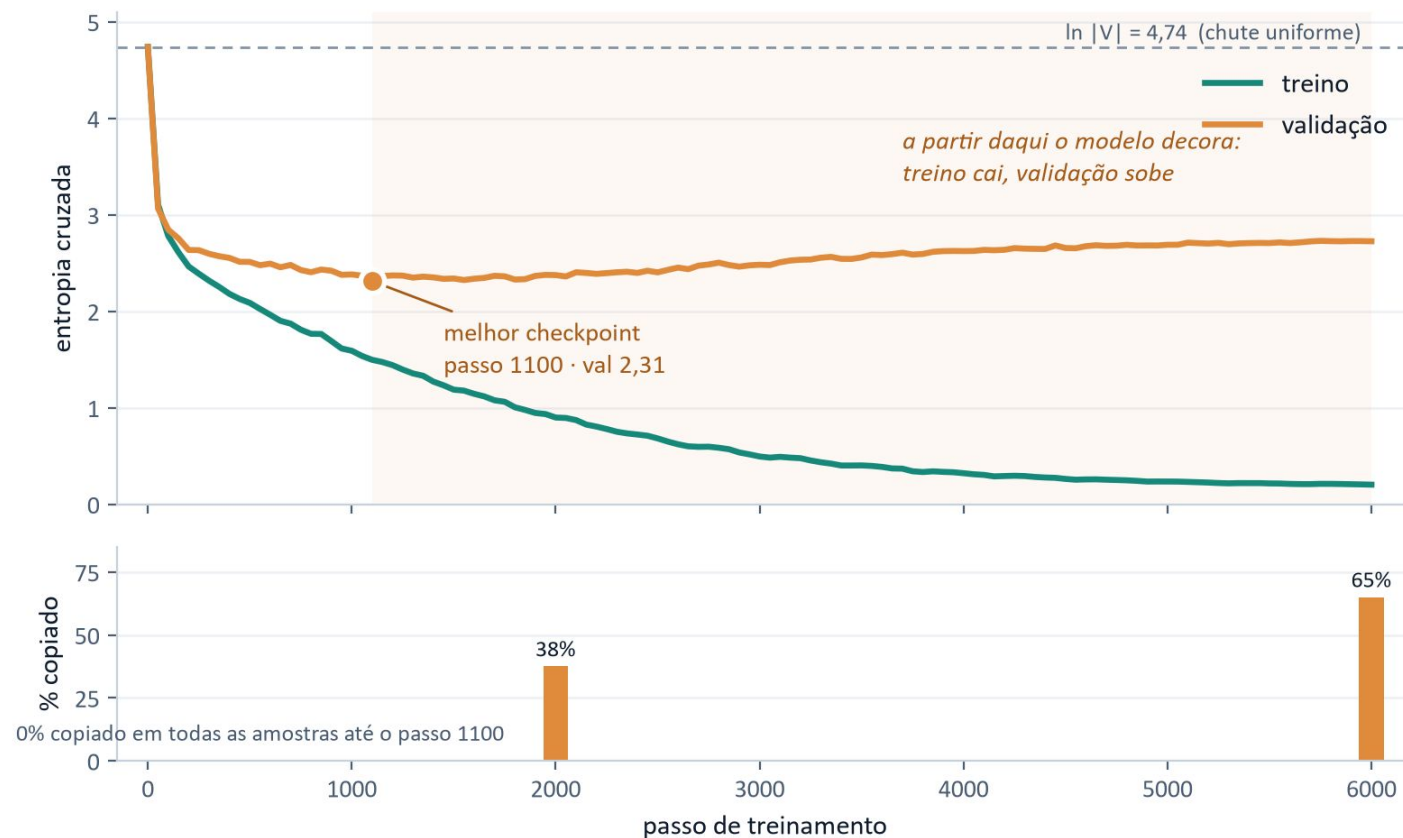
η sobe linearmente nos primeiros passos e depois decai em cosseno até 10% do pico — o esquema do GPT-3.

Otimizador Adam

Ajusta o passo de cada peso pela escala dos seus gradientes. Usamos $\beta_2 = 0,95$, como no GPT-3; nos LLMs a variante padrão é o AdamW.

Curvas de aprendizado: onde mora o diagnóstico

Treino e validação contam histórias diferentes. É na distância entre elas que se lê o que está acontecendo.



Até o passo 1100

As duas caem juntas: o modelo aprende padrões que valem também para texto que nunca viu.

Depois do passo 1100

Treino desce até 0,20; validação sobe até 2,73. O modelo passou a decorar.

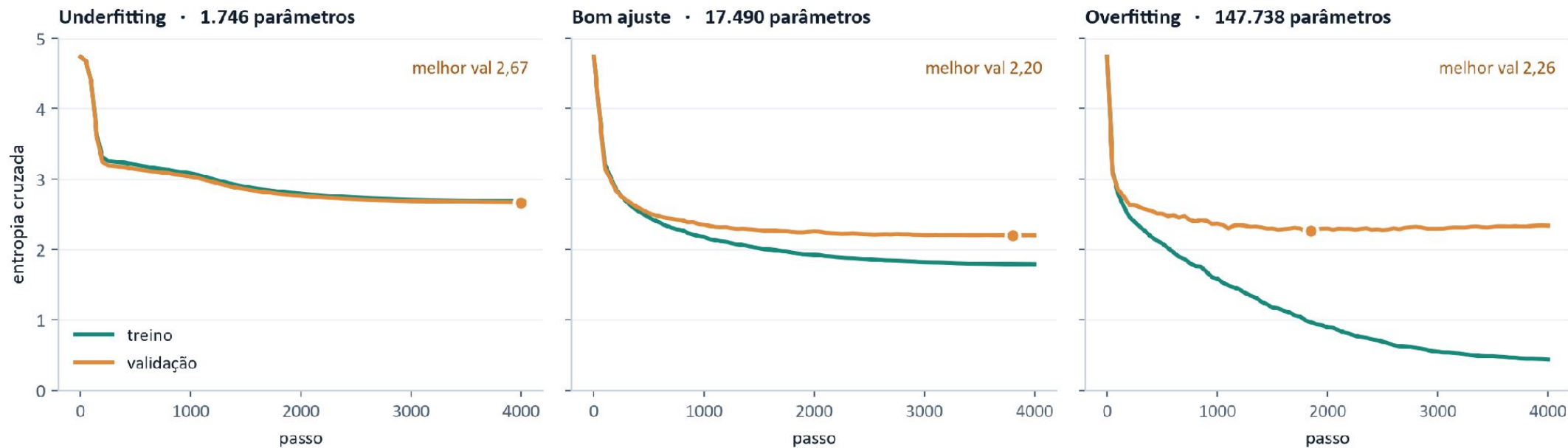
A prova da memorização

No passo 6000, 65% da amostra gerada é cópia literal do treino, com um trecho de 38 caracteres seguidos.

Por isso o checkpoint que se guarda é o do passo 1100 — não o último.

Underfitting, bom ajuste e overfitting

Mesmo corpus, mesmos 4.000 passos, mesma taxa — só muda o tamanho do modelo.



Underfitting

Treino e validação altos e colados. O modelo é pequeno demais para o padrão.

remédio: mais capacidade, mais passos

Bom ajuste

A menor validação das três (2,20), com distância pequena e estável entre as curvas.

o ponto de equilíbrio para estes dados

Overfitting

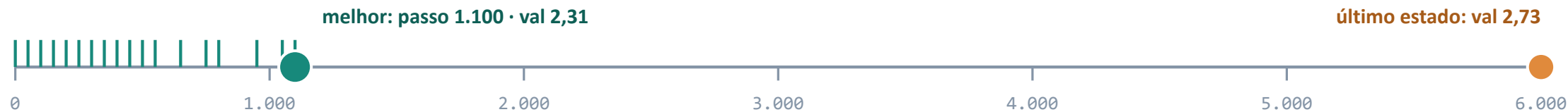
Treino desaba, validação estaciona e sobe.

remédio: mais dados, regularização, parar antes

Em LLMs grandes o overfitting é raro no pretraining: quase todo token é visto uma vez só. Repetir dados até ~4 épocas quase não prejudica (Muennighoff et al., 2023).

Checkpoints: uma fotografia completa do treino

Um checkpoint guarda o suficiente para continuar o treino como se nada tivesse acontecido.



cada traço verde é uma vez em que a validação melhorou e o checkpoint foi sobrescrito: 18 vezes, todas até o passo 1.100 — e nenhuma nas 98 avaliações seguintes

O QUE VAI DENTRO, NUM TREINO DE VERDADE

pesos do modelo

1 × o tamanho do modelo

estado do Adam (m e v)

mais 2 × o tamanho do modelo

passo e taxa atuais

para retomar o cronograma

sementes e posição nos dados

para não repetir nem pular lotes

Falhas acontecem

O treino do OPT-175B foi retomado de checkpoints dezenas de vezes por falhas de hardware (Zhang et al., 2022).

Picos de perda

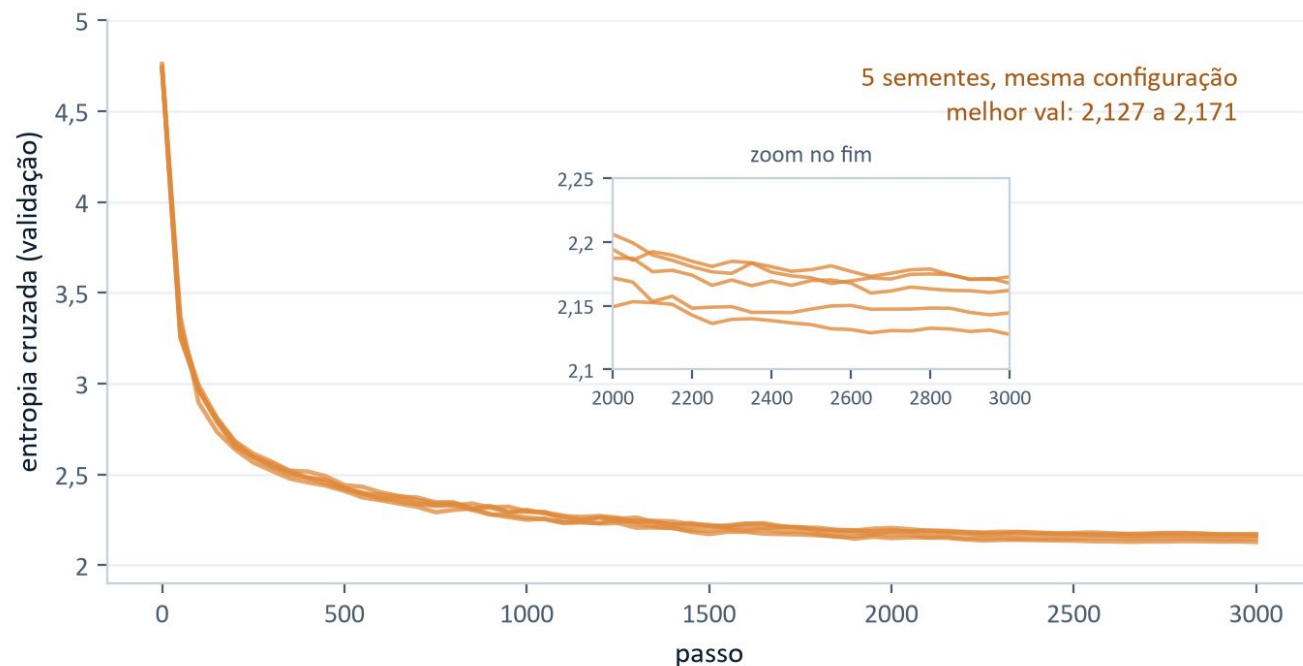
No PaLM, quando a perda disparava, a equipe voltava a um checkpoint ~100 passos antes e pulava de 200 a 500 lotes (Chowdhery et al., 2022).

Escolher o melhor

No nosso treino, o checkpoint do passo 1.100 vale mais que o do passo 6.000 — por 0,41 de perda de validação.

Sementes: quanto do resultado é sorte?

A mesma receita com sementes diferentes produz modelos diferentes. Medimos quanto.



O que a semente controla

A inicialização dos pesos, a ordem em que os lotes são sorteados, o dropout e a amostragem na hora de gerar texto.

O tamanho do ruído

5 sementes: melhor validação entre 2,127 e 2,171.

Desvio-padrão de 0,016.

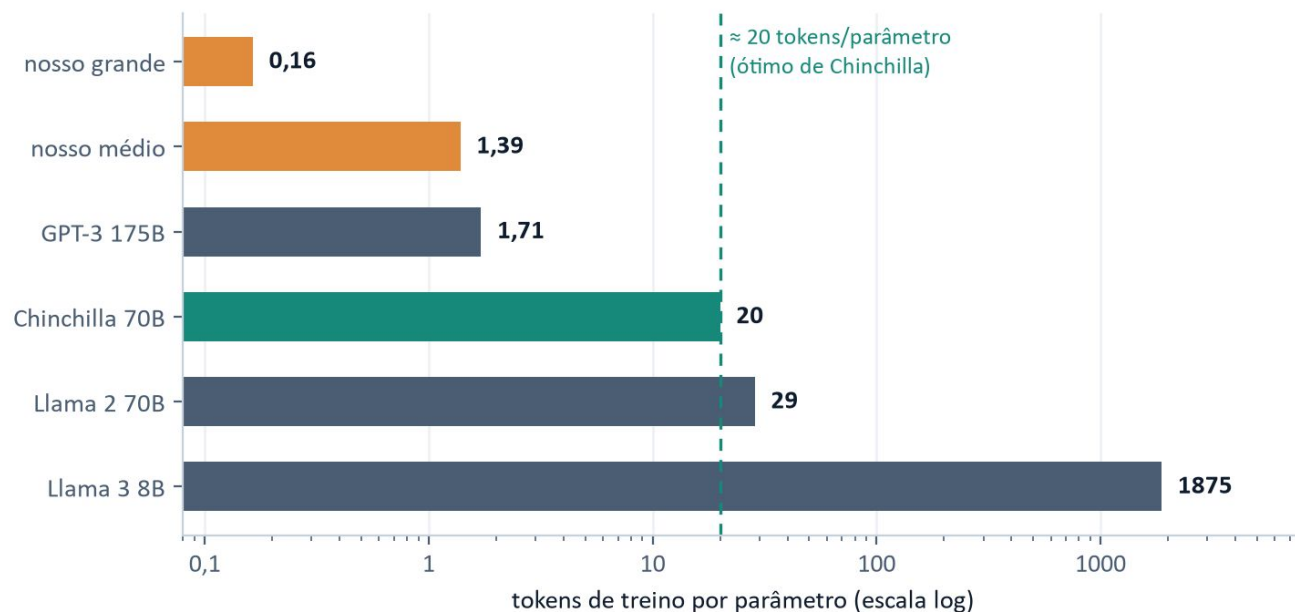
A regra prática

Uma diferença menor que o ruído entre sementes não é resultado: é sorte. Compare médias de várias sementes — em modelos pequenos, onde isso é barato.

Orçamento: toda decisão divide a mesma conta

$$C \approx 6 \cdot N \cdot D$$

computação $\approx 6 \times \text{parâmetros} \times \text{tokens}$ (Kaplan et al., 2020)



A regra de Chinchilla

Para um orçamento fixo, o ótimo fica perto de 20 tokens por parâmetro (Hoffmann et al., 2022). O GPT-3, com 1,7, era grande demais para os dados que viu.

Por que o Llama 3 passa de 1.800

Treinar demais um modelo pequeno custa caro uma única vez, mas barateia cada uso depois. O orçamento inclui a inferência.

E o nosso?

O modelo grande tinha 0,16 token por parâmetro — mais de cem vezes abaixo de Chinchilla. É a explicação quantitativa de por que ele decorou.

Tokens de treino: GPT-3 300 bi · Chinchilla 1,4 tri · Llama 2 2 tri · Llama 3 15 tri. Nosso: tokens únicos de treino.

4

Visualização e artefato

O modelo que gerou todos os números deste seminário.

- O notebook
- O que mostrar ao vivo

5

Literatura e projeto

De onde vêm as regras — e para onde elas levam.

- Artigos de referência

Literatura de referência

Bengio et al. (2003)

nosso modelo

A Neural Probabilistic Language Model

A arquitetura do nosso modelo: embeddings e uma camada oculta.

Kaplan et al. (2020)

escala

Scaling Laws for Neural Language Models

A perda cai como lei de potência em N, D e C. Origem de $C \approx 6ND$.

Raffel et al. (2020)

limpeza

Exploring the Limits of Transfer Learning (T5)

O corpus C4 e as heurísticas de limpeza que aplicamos.

Muennighoff et al. (2023)

épocas

Scaling Data-Constrained Language Models

Repetir dados até ~4 épocas quase não prejudica a perda.

Brown et al. (2020)

GPT-3

Language Models are Few-Shot Learners

GPT-3: a mistura de dados, o cronograma da taxa e o custo citados aqui.

Hoffmann et al. (2022)

orçamento

Training Compute-Optimal Large Language Models

Chinchilla: cerca de 20 tokens por parâmetro.

Lee et al. (2022)

deduplicação

Deduplicating Training Data Makes Language Models Better

Deduplicar faz o modelo emitir texto memorizado 10 vezes menos.

Loshchilov & Hutter (2019)

otimização

Decoupled Weight Decay Regularization

AdamW, o otimizador padrão do pretraining de LLMs.

Também citados: Chowdhery et al. (2022, PaLM) e Zhang et al. (2022, OPT), pelos relatos de checkpoints e picos de perda.

Números de “nosso modelo”: medição própria, notebook que acompanha este deck.

Obrigado

Perguntas?

passo 0 `2cn%4Σ+1—oMbV A3Çtbàf\8f_Nl<Íé1YB,ôT3s;ô: %:`

passo 2000 token model uma frase e com o valor um LLM e c

a mesma rede, 2.000 correções depois

Equipe • Treinamento e fine-tuning

Gabriel, Jolie e Matheus

Notebook, figuras e logs de treino
acompanham este deck.